



By: Group 2

BFS ROUTE FINDER



Tugas Besar Struktur Data



MEMBER OF GROUP:

Favian Arkaanda(124450021)

M. Razan Maulana Pratama(124450031)

Riska Erlis Dayu Tiara(124450022)

Hani Qurrota Aini(124450020)

Farhanah Hadaya Fatin (124450026)

Charrlindah(124450037)



LATAR BELAKANG

Jaringan penerbangan dunia sangat luas dan kompleks, Banyak bandara dan rute saling terhubung Penumpang membutuhkan rute paling efisien dengan transit minimum

- **Permasalahan**

Sulit menentukan jalur penerbangan terbaik, Dibutuhkan sistem pencarian rute yang cepat dan efisien

- **Pemodelan Masalah**

Bandara direpresentasikan sebagai node, Rute penerbangan direpresentasikan sebagai edge, Permasalahan dimodelkan dalam bentuk graf

- **Solusi yang Digunakan**

Menggunakan algoritma Breadth-First Search (BFS), BFS cocok untuk mencari jalur dengan transit paling sedikit

- **Cara Kerja BFS**

Menelusuri graf secara bertahap dari bandara asal, Jalur pertama yang ditemukan merupakan jalur terpendek berdasarkan jumlah transit

- **Implementasi**

Menggunakan struktur data Queue, Membantu proses pencarian berjalan sistematis dan efisien

KONSEP DASAR

Queue (FIFO)

Struktur data linear yang menerapkan prinsip First-In-First-Out. Elemen yang masuk pertama akan keluar pertama.

Enqueue: Tambah di belakang
Dequeue: Ambil di depan

BFS Traversal

Algoritma penelusuran graf secara Level-by-Level. Mengunjungi semua tetangga terlebih dahulu sebelum ke level berikutnya.

1. Mulai dari node asal
2. Enqueue tetangga
3. Dequeue & Proses



STRUKTUR KODE

DataLoader

- **load_data()** Muat & parse file CSV
- **build_graph()** Bangun adjacency list
- **get_airport_list()** Daftar semua bandara

Queue

- **enqueue(item)** Tambah ke belakang
- **dequeue()** Ambil dari depan (FIFO)
- **gis_empty()** Cek queue kosong

BFS Shortest Path

- **find_shortest_path()** Cari rute terpendek BFS
- **analyze_reachability()** Analisis jangkauan per level
- **check_connectivity()** Cek koneksi 2 bandara

IMPLEMENTASI KODE

QUEUE

```
class Queue:
    """Implementasi Queue sederhana
    untuk BFS."""
    def __init__(self):
        self._queue: deque = deque()

    def enqueue(self, item) -> None:
        """Menambahkan item ke akhir
        queue."""
        self._queue.append(item)
```

```
    def dequeue(self):
        """Mengambil dan menghapus item
        dari depan queue."""
        if self.is_empty():
            return None
        return self._queue.popleft()
    def is_empty(self) -> bool:
        """Memeriksa apakah queue kosong."""
        return len(self._queue) == 0

    def size(self) -> int:
        """Mengembalikan ukuran queue."""
        return len(self._queue)
```

- ➡ **enqueue():** Menambahkan item ke **belakang** queue menggunakan `append()`
- ➡ **dequeue():** Mengambil item dari **depan** queue menggunakan `popleft()`
- ➡ **is_empty():** Memeriksa apakah queue kosong untuk kontrol loop BFS
- ➡ **Prinsip:** FIFO (First In, First Out)
Menggunakan `collections.deque` untuk $O(1)$ enqueue & dequeue



IMPLEMENTASI KODE

BFS

```
def find_shortest_path(self, start, end):
    if start not in self.graph or end not in self.graph:
        return None

    if start == end:
        return [start]

    self.queue = Queue()
    visited: Set[str] = set()
    parent: Dict[str, Optional[str]] = {start: None}

    self.queue.enqueue(start)
    visited.add(start)

    while not self.queue.is_empty():
        current = self.queue.dequeue()

        for neighbor in self.graph.get(current, set()):
            if neighbor not in visited:
                visited.add(neighbor)
                parent[neighbor] = current
                self.queue.enqueue(neighbor)

            if neighbor == end:
                return self._reconstruct_path(parent, start, end)
        return None

    def _reconstruct_path(self, parent, start, end):
        path = []
        current = end

        while current is not None:
            path.append(current)
            current = parent[current]

        path.reverse()
        return path
```

- ➡ **Inisialisasi:** Enqueue start node, tandai sebagai visited, set parent[start] = None
- ➡ **Proses Level-by-Level:** While queue not empty: dequeue current node, proses semua tetangga
- ➡ **Explore Neighbors:** Untuk setiap unvisited neighbor: mark visited, set parent, enqueue
- ➡ **Target Found:** Jika neighbor == target, reconstruct path dari parent dictionary
- ➡ **Prinsip:** BFS menjamin rute terpendek (jumlah transit minimum) karena menelusuri secara level-by-level





FUNGSI ANALISIS

analyze_reachability_by_level()

```
def analyze_reachability_by_level(self, start, max_levels=5):  
    levels = {i: [] for i in range(max_levels + 1)}  
    visited = set()  
    queue.enqueue((start, 0)) # (node, level)  
    while not queue.is_empty():  
        current_node, current_level = queue.dequeue()  
        next_level = current_level + 1  
        for neighbor in graph.get(current_node):  
            if neighbor not in visited:  
                levels[next_level].append(neighbor)  
                queue.enqueue((neighbor, next_level))
```

Queue menyimpan (node, level) tuple

check_connectivity()

```
def check_connectivity(self, start, end):  
    path = self.find_shortest_path(start, end)  
    is_connected = path is not None  
    # Hitung bandara terjangkau dari start  
    reachable_count = 0  
    visited = set()  
    queue.enqueue(start)  
    while not queue.is_empty():  
        node = queue.dequeue()  
        reachable_count += 1  
        for neighbor in graph.get(node):  
            if neighbor not in visited:  
                visited.add(neighbor)  
                queue.enqueue(neighbor)  
    return is_connected, path, transits, reachable_count
```

Menggunakan find_shortest_path() untuk cek koneksi
Menghitung reachable_count dari bandara asal

DATALOADER CLASS

load_data():

Membaca dan mem-parse file CSV

```
def load_data(self): with open(self.filepath, 'r') as file: content = file.read()
lines = content.split('\n') reader = csv.DictReader(lines) for row in reader:
source = row[' source airport'].strip() destination = row[' destination
airport'].strip() self.routes.append({...}) self.airports.add(source)
self.airports.add(destination)
```

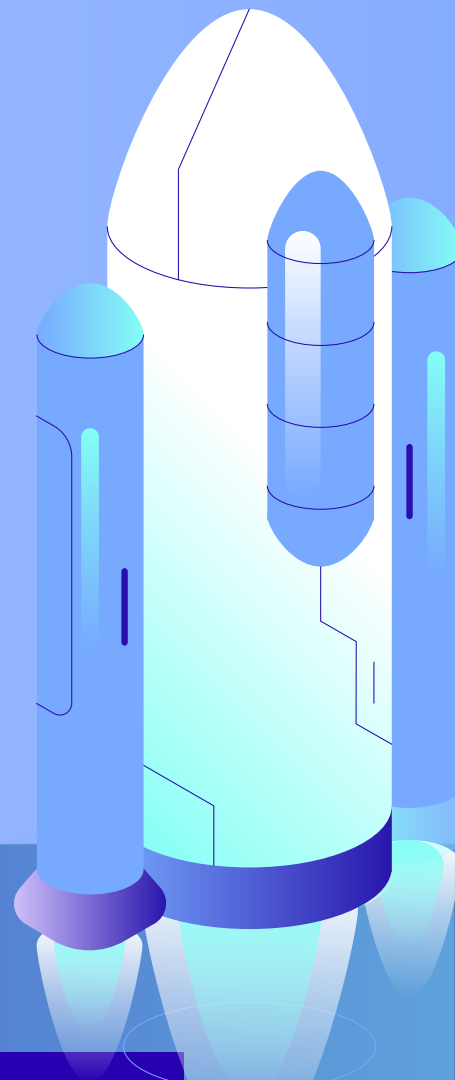
- Membaca routes.csv dengan DictReader

build_graph():

Membangun adjacency list graph

```
def build_graph(self): self.graph = {airport: set() for airport in
self.airports} for route in self.routes: source = route['source']
destination = route['destination']
self.graph[source].add(destination)
```

- Graph = Dict[str, Set[str]]
- Key = Kode bandara (IATA)
- Value = Set destinasi langsung



PATH RECONSTRUCTION & FLIGHT DETAILS

`_reconstruct_path():`

Merekonstruksi jalur dari parent dict

```
def _reconstruct_path(self, parent, start, end):  
    path = []  
    current = end  
    while current is not None:  
        path.append(current)  
        current = parent[current]  
    path.reverse()  
    return path
```

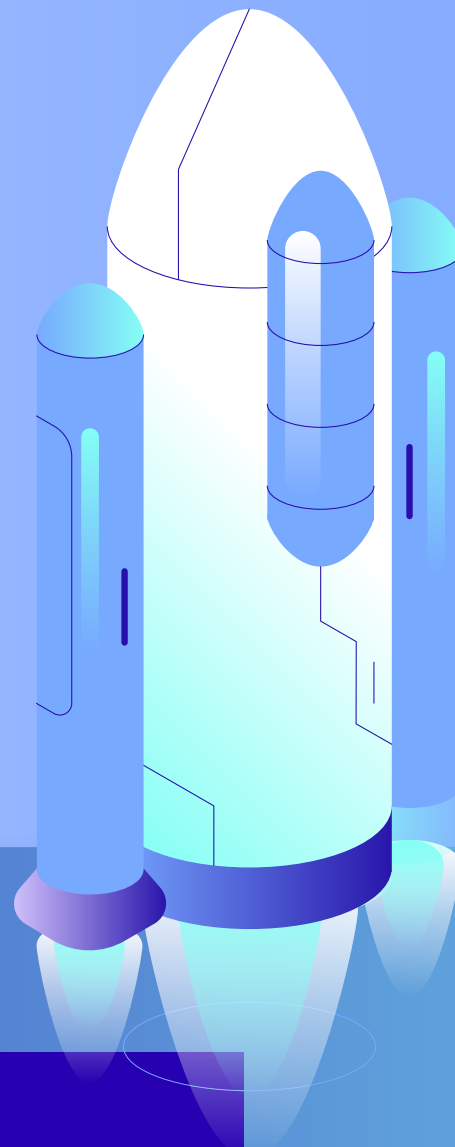
- Mulai dari end, backtrack ke start
- Gunakan **parent[current]** untuk navigasi
- **reverse()** untuk mendapatkan start → end

`get_flight_details():`

Mendapatkan detail penerbangan per segmen

```
d def get_flight_details(self, path):  
    details = []  
    for i in range(len(path) - 1):  
        source = path[i]  
        destination = path[i + 1]  
        for route in self.data_loader.routes:  
            if route['source'] == source and \ route['destination'] == destination:  
                details.append(route)  
                break  
    return details
```

- Iterasi setiap segmen (pair berurutan) str, Set[str]
- Cari rute yang cocok di self.routes
- Return list detail: airline, equipment, dll



KOMPLEKSITAS & EFISIENSI

Time Complexity

$$O(V + E)$$

Vertices + Edges

- V = Vertices (3,425 Bandara)
- E = Edges (67,663 Rute)
- Setiap node & edge dikunjungi sekali

Space Complexity

$$O(V)$$

Linear Space for Visited & Parent

- visited set (track processed nodes)
- parent dict (reconstruct path)
- queue (max size $O(V)$ at worst)

BFS vs DFS untuk Shortest Path

BFS:

- Traversal: Level-by-level
- Guarantee: Shortest path (unweighted)
- Uses: Queue (FIFO)

DFS:

- Traversal: Depth-first
- Guarantee: No guarantee shortest
- Uses: Stack (LIFO)

IMPLEMENTASI & DATA PENERBANGAN



TOTAL RUTE

67,663

Koneksi penerbangan langsung



TOTAL BANDARA

3,425

Node dalam graf (Worldwide)

Representasi Graf

Nodes (Simpul): Bandara (IATA Code)

Edges (Sisi): Rute Penerbangan Langsung

Weight: Jumlah Transit (1 hop = 1)

Struktur data Utama

Graph: Adjacency List

Queue: BFS Frontier

Map: Parent Tracking





USE CASE: RUTE LANGSUNG



Pencarian rute tanpa transit (0 Stops)

KZN  **LED** CASE 1

Region Rusia

Maskapai 2B

Equipment CR2

SIN  **BKK** CASE 2

Region Asia Tenggara

Maskapai 3K

Equipment 320

MNL  **KIX** CASE 3

Region Asia

Maskapai 3K

Equipment 320





Use Case: Rute Transit

Pencarian rute dengan 1 kali transit (1 Stop)

1 TRANSIT

CASE 4

CEK —  — **DME** —  — **NBC**
START TRANSIT END

→ Segmen 1: CEK -> DME

Maskapai: S7

Equipment: 319

→ Segmen 2: DME -> NBC

Maskapai: 2B

Equipment: CR2

1 TRANSIT

CASE 5

ACH —  — **EBA** —  — **MUC**
START TRANSIT END

→ Segmen 1: ACH -> EBA

Maskapai: 3L

Equipment: DH3

→ Segmen 2: EBA -> MUC

Maskapai: 3L

Equipment: DH3

1 TRANSIT

CASE 6

FLL —  — **DCA** —  — **PNS**
START TRANSIT END

→ Segmen 1: FLL -> DCA

Maskapai: AA

Equipment: 319

→ Segmen 2: DCA -> PNS

Maskapai: AA

Equipment: CRJ



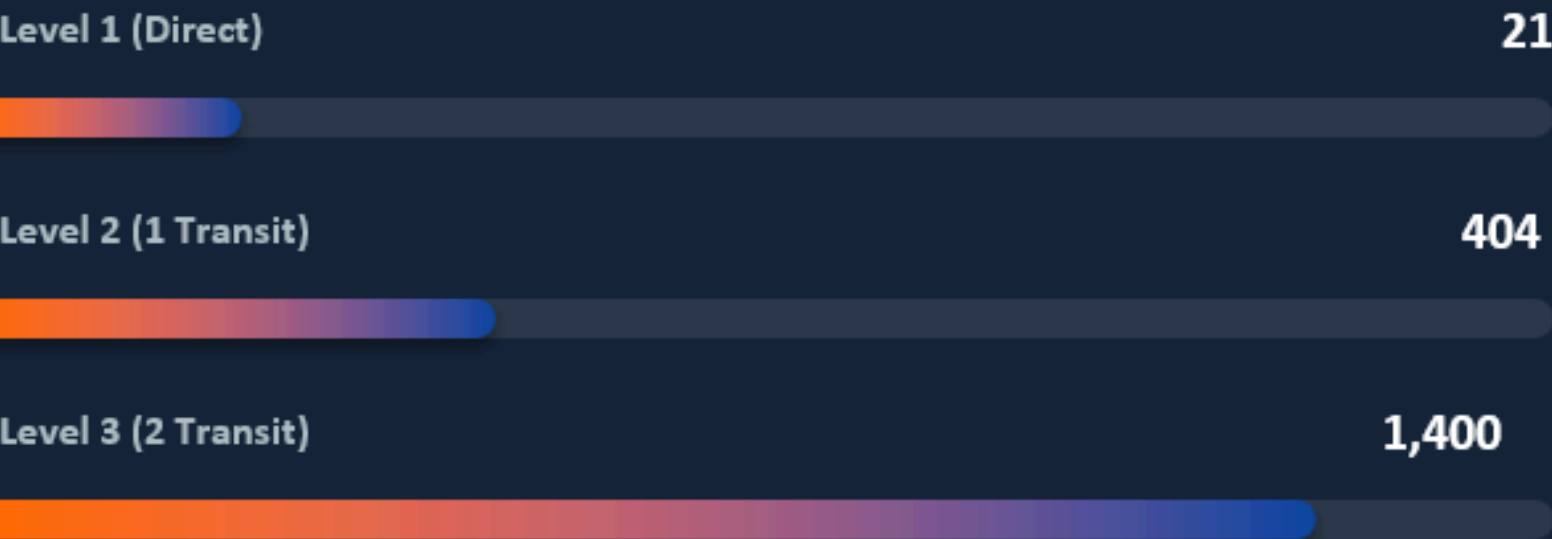
Analisis Jangkauan Per Level

Perbandingan konektivitas bandara berdasarkan level transit (BFS Level)

KZN

Kazan Airport, Rusia

✈️ Minor Hub



Jangkauan langsung terbatas (21 bandara). Membutuhkan 2-3 transit untuk mencapai sebagian besar jaringan global.

SIN

Changi Airport, Singapura

🌐 Major Hub



Super Connector: 125 rute langsung! Hanya butuh 1 transit untuk mencapai >1,200 bandara. Konektivitas tingkat dunia.



Kesimpulan

Implementasi Queue + BFS pada Pencarian Rute Penerbangan



Pencarian Rute Optimal

Algoritma BFS dengan Queue menjamin penemuan **route terpendek** (jumlah transit minimum) pada graf tidak berbobot. Setiap level traversal merepresentasikan jumlah transit.



Konektivitas Jaringan

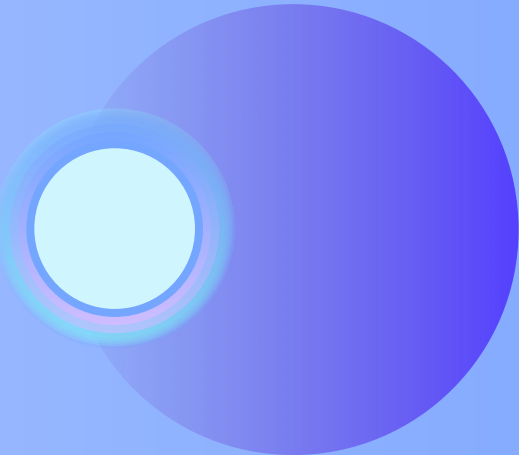
Analisis menunjukkan mayoritas bandara terhubung dalam **2-3 level transit**. Hub besar seperti SIN memiliki jangkauan luas (125 langsung), sementara KZN lebih terbatas (21 langsung).



Efisiensi Algoritma

Struktur Queue memastikan eksplorasi **level-by-level** yang sistematis. Kompleksitas $O(V+E)$ cocok untuk dataset besar dengan 3,425 node dan 67,663 edges.





THANK YOU!